

Synthesizable and Non-Synthesizable Constructs

Introduction

This lab introduces the distinction between synthesizable and non-synthesizable constructs in Verilog, using Yosys – an open-source synthesis tool. You will design simple hardware modules incorporating both types of constraints, explore how Yosys interprets them, and understand which constructs are suitable for synthesis.

Through practical experiments, you will write Verilog code, run synthesis with Yosys and observe how unsupported or simulation-only constructs are treated during the synthesis process. This lab helps build a strong foundation for RTL design best practices and improves your ability to write clean, synthesizable hardware descriptions.

Objectives

After completing this lab, you will be able to:

- Differentiate between synthesizable and non-synthesizable constructs in Verilog.
- Write Verilog code using both types of constructs for experimental analysis.
- Synthesize Verilog designs using Yosys and identify unsupported/non-synthesizable elements.
- Understand how certain constraints and constructs affect simulation vs synthesis.
- Understand modular hardware integration using synthesizable RTL design.

Procedure:

This lab consists of the following steps:

1. Creating the lab directory
2. Creating new project directory
3. Writing the design files
4. Writing the yosys synthesis script file
5. Synthesis of the design using Yosys
6. Observing the difference between synthesizable and non-synthesizable Verilog code.

Design 1: And Gate Synthesizable Verilog code

Steps:

Step 1: Create a Lab Directory

1.1: Create a new directory inside your lab workspace:

```
mkdir synthesis_lab
```

1.2: Navigate to the lab directory:

```
cd synthesis_lab
```

1.3: Create a new project directory for and gate verilog code.

```
mkdir and_gate
```

1.4: Navigate into the project directory

```
cd and_gate
```

Lab 1: Synthesis of and gate design

Step 1: Create the Design File

Open GVim to write the synthesizable and gate code

```
gvim and_gate.v (Synthesizable design code)
```

// Synthesizable and_gate design code:

```
module and_gate(input a, input b, output y);  
    assign y = a & b;  
endmodule
```

Step 2: Create the Yosys Script File

3.1: Open GVim to create the yosys script file.

```
gvim and_gate.ys
```

3.2: Write the yosys script file for doing the synthesis

#Yosys Script File

```
read_verilog and_gate.v
```

#(This command reads the Verilog Input File named and_gate.v)

```
hierarchy -check -top and_gate
```

#(This command checks the hierarchy of the design from the top with the top-level module name as and_gate)

```
proc; opt; fsm; opt; memory; opt
```

#(This command performs the internal processes done by yosys tool as well as map the fsm's, and memory block if any are present in the design)

```
techmap; opt
```

#(This command maps the design to your internal cell library)

```
dfflibmap -liberty /home/<coe-
```

```
no>/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
```

#(This command maps the flip-flops present in your design to your standard cell library)

```
abc -liberty /home/<coe-
```

```
no>/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
```

#(This command maps the logic of your design to your standard cell library)

```
clean
```

#(This command removes the unused wires if any from your design)

show

#(This command opens a pictorial representation of your design showing how many standard cells are used and how they are connected in your design)

write_verilog and_gate_netlist.v

#(This command writes the synthesized gate level netlist in Verilog format with the name and_gate_netlist.v)

3.3: Save and exit the editor

Type :wq! and press Enter

3.4: List the files to confirm they are saved : ls

Step 4: Synthesize the Design using Yosys

4.1: Run Yosys Open-Synthesis Suit Tool

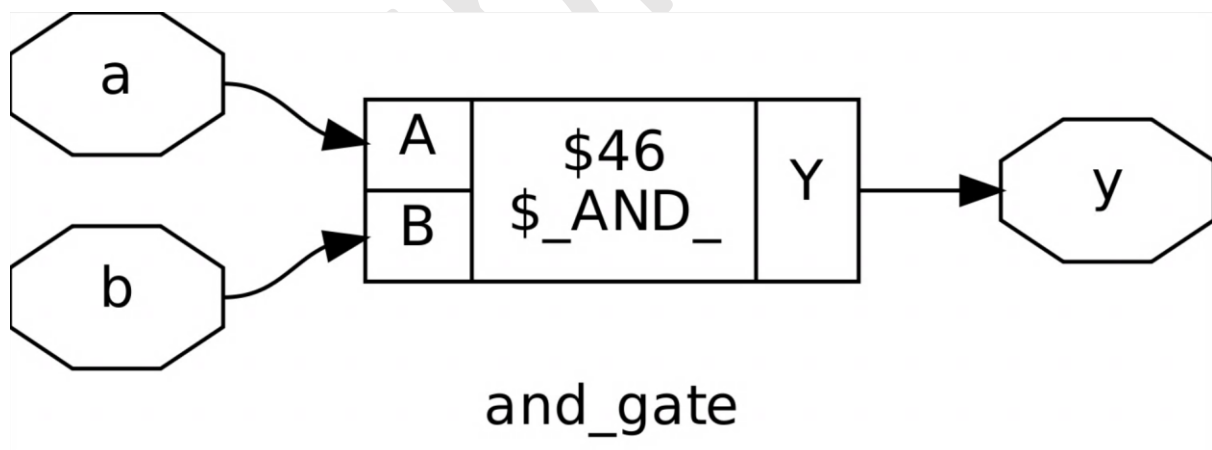
yosys

4.2: Now run the script file you have created for the design

script and_gate.y

Step 5: View Synthesis Output

5.1: The synthesis will pass for the synthesizable Verilog code.



5.2: Now exit from the Yosys tool

exit

5.3: After successful synthesis, a new file i.e. synthesized gate level netlist and_gate_netlist.v will be created.

5.4 Observe the difference between and_gate.v and and_gate_netlist.v

Lab 2 : Synthesis of And_Gate non-synthesizable design code

Step 1: Navigate into the project directory: and_gate

Step 2: Open GVim to write the non-synthesizable and gate design code

```
gvim and_gate_non_synth.v (non-synthesizable design code)
```

```
// Non-Synthesizable AND gate design code
```

```
module and_gate_non_synth (  
    input a,  
    input b,  
    output reg y  
);  
  
// Non-synthesizable construct: Initial block  
initial begin  
    y = 0;  
end  
  
// Non-synthesizable construct: Delay (#)  
always @(a or b) begin  
    #5 y = a & b;  
end  
  
// Non-synthesizable construct: $display system task  
always @(y) begin  
    $display("TIME=%0t : a=%b b=%b y=%b", $time, a, b, y);  
end  
  
// Non-synthesizable construct: $monitor task  
initial begin  
    $monitor("MONITOR: time=%0t a=%b b=%b y=%b", $time, a, b, y);  
end  
// Non-synthesizable construct: Simulation termination  
initial begin  
    #120;  
    $display("Simulation Finished");  
    $finish;  
end  
  
endmodule
```

2.3: Save and exit the editor

Type :wq! and press Enter

Step 3: Create the Yosys Script File

3.1: Open GVim to create the yosys script file.

```
gvim and_gate_non_synth.ys
```

3.2: Write the following yosys synthesis script

#Yosys Script File

```
read_verilog and_gate_non_synth.v
#!/( This command reads the Verilog input file)

hierarchy -check -top and_gate_non_synth
#!/(This command checks the hierarchy of the design and sets and_gate_non_synth as the top module)

proc; opt; fsm; opt; memory; opt
#!/(This command performs the internal processes done by yosys tool as well as map the fsms, and memory block if any are present in the design)

techmap; opt
#!/(This command maps the design to your internal cell library)

dfflibmap -liberty /home/<coe-
no>/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
#!/(This command maps the flip-flops present in your design to your standard cell library)

abc -liberty /home/<coe-
no>/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
#!/(This command maps the logic of your design to your standard cell library)

clean
#!/(This command removes the unused wires if any from your design)

show
#!/(This command opens a pictorial representation of your design showing how many standard cells are used and how they are connected in your design)

write_verilog and_gate_netlist_non_synth.v
#!/(This command writes the synthesized gate level netlist in Verilog format with the name and_gate_netlist_non_synth.v)
```

3.3: Save and exit the editor

Type : wq and press Enter

3.4: List the files to confirm they are saved: ls

Step 4: Synthesize the Design using Yosys

4.1: Run Yosys Open-Synthesis Suit Tool

```
yosys
```

4.2: Now run the script file you have created for the design

```
script and_gate_non_synth.ys
```

Step 5: View Synthesis Output

5.1: The synthesis will pass for the synthesizable Verilog code.

```
Generating RTLIL representation for module ``and_gate_non_synth'.  
and_gate_non_synth.v:23: Warning: System task '$display' outside initial block is unsupported.  
and_gate_non_synth.v:28: Warning: Ignoring call to system task $monitor.  
Simulation Finished  
and_gate_non_synth.v:35: ERROR: System task '$finish' executed.
```

5.2: Now exit from the Yosys tool

```
exit
```

5.3: After successful synthesis, a new file i.e. synthesized gate level netlist
and_gate_netlist_non_synth.v will be created

```
ls
```

5.4 Observe the difference between and_gate_non_synth.v and
and_gate_netlist_non_synth.v

Lab 3: Synthesis of mini_soc design

Step 1: Navigate back to the Lab Directory

1.1: Navigate to the lab directory:

```
cd ~/synthesis_lab
```

1.2: Create a new project directory for mini soc

```
mkdir mini_soc
```

1.3: Navigate into the project directory

```
cd mini_soc
```

Step 2: Create the Design File

Create design files of mini soc

1: 4-bit Counter IP

Design a synchronous 4-bit up-counter with reset, which serves as a reusable count source in the SoC.

Create the design file for counter IP : counter.v

```
module counter (  
    input clk,  
    input rst,  
    output reg [3:0] count  
);  
    always @(posedge clk) begin  
        if (rst)  
            count <= 0;  
        else  
            count <= count + 1;  
        end  
endmodule
```

2: 2-bit ALU IP (Add / Subtract)

Implement a 2-bit Arithmetic Logic Unit that performs either addition or subtraction based on a select input.

Create the file for ALU IP : `alu.v`

```
// Simple 2-bit ALU with add and subtract
module alu (
    input [1:0] a,
    input [1:0] b,
    input sel,          // 0: ADD, 1: SUB
    output reg [1:0] result
);
    always @(*) begin
        case (sel)
            1'b0: result = a + b;
            1'b1: result = a - b;
        endcase
    end
endmodule
```

3: 2x1 Multiplexer IP

Create a 2:1 multiplexer that selects between two 2-bit input signals, forwarding the selected signal to the output.

Create the file for Multiplexer IP: `mux2x1.v`

```
// 2-bit 2:1 MUX
module mux2x1 (
    input [1:0] in0,
    input [1:0] in1,
    input sel,
    output [1:0] out
);
    assign out = sel ? in1 : in0;
endmodule
```


4: SoC Top-Level Module

Integrate the counter, ALU, and MUX IP cores into a single top-level SoC module with clear interconnections.

This top-level module encapsulates the entire mini-SoC behavior and demonstrates full modular integration.

Create the file for SoC Top: `mini_soc.v`

// Integrates counter, ALU, and MUX

```
module mini_soc (  
    input clk,  
    input rst,  
    input alu_sel,  
    input mux_sel,  
    output [1:0] soc_out  
);  
    wire [3:0] count;  
    wire [1:0] alu_out;  
    // Instantiate IP blocks  
    counter u_counter (.clk(clk), .rst(rst), .count(count));  
    alu u_alu (.a(count[3:2]), .b(count[1:0]), .sel(alu_sel),  
    .result(alu_out));  
    mux2x1 u_mux (.in0(count[3:2]), .in1(alu_out), .sel(mux_sel),  
    .out(soc_out));  
endmodule
```

Step 3: Create the Yosys Script File

3.1: Open GVim to create the yosys script file.

```
gvim mini_soc.ys
```

3.2: Write the yosys script file for doing the synthesis

Yosys Script File

```
read_verilog counter.v alu.v mux2x1.v mini_soc.v
```

#(These commands read all the Verilog design files)

```
hierarchy -check -top mini_soc
```

#(This command checks the hierarchy of the design from the top with the top-level module name as mini_soc)

```
proc; opt; fsm; opt; memory; opt;
```

#(This command performs the internal processes done by yosys tool as well as maps the FSMs and memory blocks if any are present in the design)

```
techmap; opt;
```

#(This command maps the design to the internal cell library)

```
dfflibmap -liberty /home/<coe-  
no>/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
```

#(This command maps the flip-flops present in the design to the standard cell library)

```
abc -liberty /home/<coe-  
no>/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
```

#(This command maps the logic of the design to the standard cell library)

```
clean
```

#(This command removes unused wires if any from the design)

```
show
```

#(This command opens a pictorial representation of the synthesized design showing the standard cells used and their connections)

```
write_verilog mini_soc_netlist.v
```

#(This command writes the synthesized gate-level netlist in Verilog format with the name mini_soc_netlist.v)

3.3: Save and exit the editor

Type :wq! and press Enter

3.4: List the files to confirm they are saved

```
ls
```

Step 4: Synthesize the Design using Yosys

4.1: Run Yosys Open-Synthesis Suit Tool

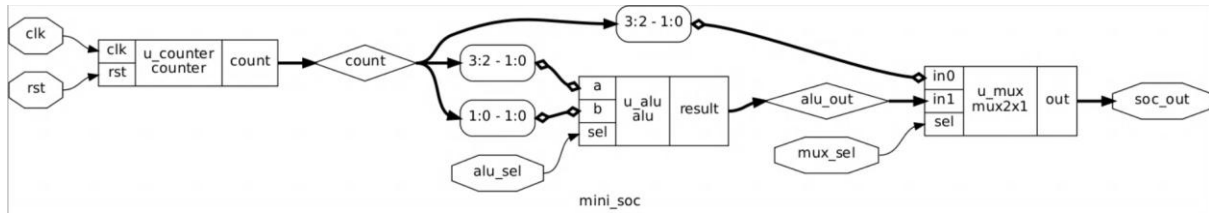
```
yosys
```

4.2: Now run the script file you have created for the design

```
script mini_soc.ys
```

Step 5: View Synthesis Output

5.1: The synthesis will pass for the synthesizable Verilog code.



5.2: Now exit from the Yosys tool

```
exit
```

5.3: After successful synthesis, a new file i.e. synthesized gate level netlist mini_soc_netlist.v will be created

```
ls
```

5.4 Observe the difference between mini_soc.v and mini_soc_netlist.v

Conclusion

In this lab, you explored the difference between synthesizable and non-synthesizable Verilog constructs using both simple and modular RTL examples.

You implemented and synthesized:

- AND gate design
- Non-synthesizable testbench analysis
- Mini-SoC integration design

Using Yosys, you observed how synthesizable RTL is transformed into gate-level hardware while simulation-only constructs are ignored or rejected during synthesis.

This lab strengthened your understanding of:

- RTL coding practices
- Synthesis-compliant Verilog
- Modular hardware design
- Hierarchical SoC integration
- Open-source synthesis workflows

Understanding synthesizable and non-synthesizable constraints is essential for writing reliable hardware descriptions and developing efficient digital systems.